

Advanced RAG — 검색 정확도를 높이는 7가지 기법

Session 2에서 기본 RAG 파이프라인을 구성했다면, Session 3에서는 검색 정확도를 높이는 주요 기법을 다룬다.
작동 원리, 공개 벤치마크, 파일럿 결과, 비용·지연, 적용 한계를 함께 비교한다.

—67%

Contextual Retrieval + Reranking
Retrieval Failure Rate 감소

+27%

Step-Back Prompting
TimeQA Accuracy

—30%p

Multi-hop RAG
Accuracy 감소 (NovelHopQA)

+23.8%p

Hybrid Search + Query Decomposition
MultiHopRAG 250문항 파일럿

SESSION 3 · AGENDA

Advanced RAG

7가지 기법, 파일럿 비교, 적용 한계

0 RAG 파이프라인 개요

기법별 적용 단계

1 Query Understanding

Query Rewriting · Query Decomposition · HyDE · Step-Back Prompting

2 Hybrid Search & RRF

Dense Retrieval · Sparse Retrieval (BM25) · RRF

3 Reranking

Cross-encoder · Cohere Rerank · LLM-as-Reranker

4 Contextual Retrieval

Contextual Embeddings · Contextual BM25

5 Metadata Filtering · Parent-Child Retrieval

Small-to-Big Retrieval · Sentence Window Retrieval

6 Multi-hop RAG와 한계

Query Decomposition · Iterative RAG · GraphRAG

7 MultiHopRAG 파일럿 평가

250문항 · Retrieval 구성과 Chunking Strategy 비교

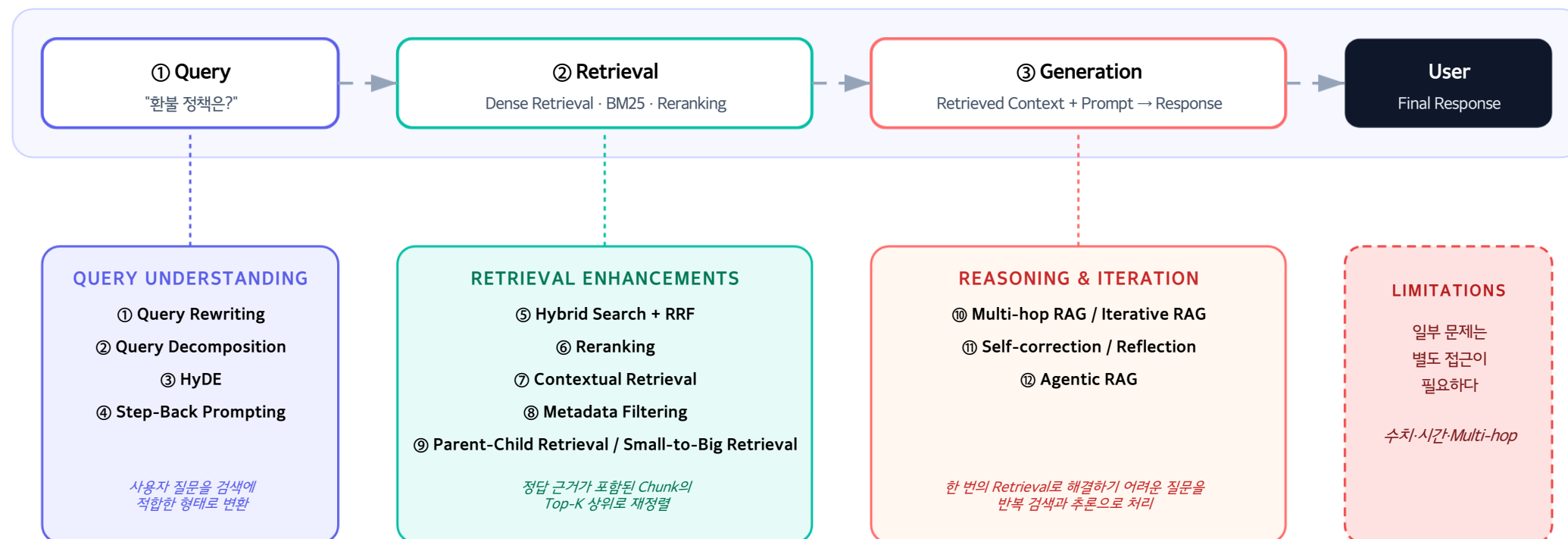


10문제 퀴즈

핵심 개념과 파일럿 결과 확인

RAG 파이프라인과 주요 기법

기법별 적용 단계



RAG 파이프라인과 주요 기법

RAG 파이프라인은 Query → Retrieval → Generation으로 구성된다. 먼저 각 기법이 적용되는 단계를 정리한 뒤, 작동 방식과 적용 조건을 살펴본다.



핵심 원칙

고급 기법은 기본 Retrieval 품질이 확보된 뒤 추가 개선을 만드는 수단이다. Chunking과 Embedding 품질을 먼저 평가하고, 확인된 약점에 맞춰 기법을 선택한다.



비용과 지연을 함께 본다

Contextual Retrieval은 인덱싱 비용을, HyDE는 Query 당 LLM 호출을, Reranker는 응답 지연을 늘린다. 품질 개선과 비용·지연을 같은 평가표에서 비교해야 한다.



기법을 조합해도 한계는 남는다

Contextual Retrieval, BM25, Reranking을 순차적으로 결합했을 때 추가 개선이 관찰됐다. 다만 Multi-hop 질문, 수치 집계, 시간 추론에는 별도 접근이 필요하다.

사용자 질문을 검색에 적합한 형태로 변환

사용자 질문은 짧거나 모호하고, 여러 요구가 한 문장에 섞여 있기도 하다. Query Rewriting, Query Decomposition, HyDE, Step-Back Prompting은 Retrieval 전에 질문을 변환해 Recall을 높이는 방법이다. 추가 LLM 호출이 발생하므로 질문 유형에 따라 선택적으로 적용한다.

검색에 불리한 질문 패턴과 대응 기법

1. 모호한 질문

"환불 어떻게?"

BM25 Top-3
"회원가입 어떻게" · "환불 변천사"
정답 문서는 Top-10 밖

해결: ① Query Rewriting

"환불 절차 · 환불 조건 ·
환불 기간 · 환불 수단"

→ 검색어를 구체화해 BM25 Recall 개선

2. 복합 질문

"Apple-Google의 2024 매출과
직원 수 차이는?"

한 번의 Retrieval: Apple 관련 Chunk에 편중
→ 필요한 4개 근거 중 2개만 검색

해결: ② Query Decomposition

Subquery × 4 (회사 × 지표)
병렬 Retrieval → Final Synthesis

→ 필요한 근거를 각각 검색

3. 짧은 질문

"양자 컴퓨팅 원리"

짧은 Query Embedding ↔ 긴 Chunk Embedding
표현 길이와 문체 차이
→ 관련 Chunk가 Top-K에서 누락

해결: ③ HyDE

LLM이 Hypothetical Document 생성
Hypothetical Document를 Embedding해 Corpus 검색

→ 문서와 유사한 표현으로 Retrieval

4. 지나치게 구체적인 질문

"Estella Leopold는 1954년
8~11월 어느 학교?"

연도와 월이 함께 등장하는 Chunk 없음
→ 직접 일치 결과 없음

해결: ④ Step-Back Prompting

시점 조건을 완화: "Estella의 학력 이력은?"
→ 학력 이력 관련 Chunk 검색

→ LLM이 연도 구간을 추론해 Yale 도출



① Query Rewriting

LLM이 질문을 검색 엔진에 적합한 표현으로 다시 작성한다. 동의어를 보완하고, 모호한 대상을 명확히 하며, 필요한 경우 여러 검색어로 확장한다. Query당 LLM 호출 1회가 추가된다.

예시 — 사내 위키에서 "환불 어떻게?" 검색

[STEP 1] Input Query

Q = "환불 어떻게?"

[STEP 1a] Input Query로 BM25 Retrieval

Top-3 by BM25 on Input Query:

- ▶ "회원 가입 어떻게 하나요?" (FAQ #12) ← '어떻게' 키워드 매치
- ▶ "환불 정책의 변천사" (블로그) ← '환불' 매치, 정책 문서 아님
- ▶ "쿠폰 환불 사례" (커뮤니티) ← 정책 문서 아님

정답 문서("환불 신청 절차 v3.2")는 Top-10 밖.

[STEP 2] Haiku로 Query Rewriting

prompt: "Rewrite this user query as 3-5 keyword-rich search queries that an internal wiki search engine would match. Original: '환불 어떻게?'"



① Query Rewriting

LLM이 질문을 검색 엔진에 적합한 표현으로 다시 작성한다. 동의어를 보완하고, 모호한 대상을 명확히 하며, 필요한 경우 여러 검색어로 확장한다. Query당 LLM 호출 1회가 추가된다.

예시 — 사내 위키에서 "환불 어떻게?" 검색

```
# [STEP 2 output]
rewrites = [
  "환불 신청 절차",
  "환불 가능 조건 · 환불 대상 품목",
  "환불 처리 기간 · 환불 수단(카드/계좌)",
  "구매 취소 vs 반품 차이"
]

# [STEP 3] 각 Rewrite를 검색하고 RRF로 결합
Top-3 after Query Rewriting + RRF:
▶
"환불 신청 절차 v3.2" (정답 문서) ★
▶ "환불 가능 품목 리스트"
▶ "환불 처리 기간 FAQ"

# [STEP 4] Top-3 Chunk로 Final Response 생성
```



① Query Rewriting

LLM이 질문을 검색 엔진에 적합한 표현으로 다시 작성한다. 동의어를 보완하고, 모호한 대상을 명확히 하며, 필요한 경우 여러 검색어로 확장한다. Query당 LLM 호출 1회가 추가된다.

벤치마크 결과

- ✓ LLM 호출 1회로 적용 가능 (~150ms / ~\$0.0005)
- ✓ BM25와 특히 잘 맞음 — 짧고 모호한 질문에 검색어 보완
- ✓ 간단한 Prompt Template로 구현 가능

한계와 주의점

- ✗ LLM이 질문 의도를 잘못 해석하면 부정확한 검색어가 생성될 수 있음
- ✗ 이미 구체적인 질문에서는 효과가 제한적
- ✗ Rewritten Query 수만큼 Retrieval 비용 증가

구현

LangChain MultiQueryRetriever

LlamaIndex QueryTransform · HyDEQueryTransform

Custom Prompt Template



② Query Decomposition

복합 질문을 독립적으로 검색할 수 있는 2~5개의 Subquery로 나눈다. 각 Subquery를 검색한 뒤 결과를 통합해 최종 답변을 생성한다.

예시 — Apple과 Google의 2024년 매출·직원 수 비교

STEP 1 · Input Query

Input Query Q

"Apple과 Google의 2024년
매출과 직원 수 차이는?"

Single Retrieval

Top-6: Apple 4개, Google 2개로 편중
직원 수 근거는 별도 IR 문서에 있어 검색되지 않음
→ 필요한 4개 근거 중 2개만 확보

STEP 2 · Query Decomposition

Q를 독립적인 Subquery로 분해

2~5개의 Subquery 생성
각 Subquery는 하나의 사실만 묻도록 제한

Subquery 1 "Apple의 2024 FY 총 매출은?"

▸ Apple 10-K: "Net sales **\$391.0B** (FY24)"

Subquery 2 "Google(Alphabet)의 2024 FY 총 매출은?"

▸ Alphabet 10-K: "Revenue **\$350.0B** in 2024"

Subquery 3 "Apple의 2024년 정규직원 수는?"

▸ Apple Form 10-K: "Full-time employees: **164,000**"

Subquery 4 "Alphabet의 2024년 정규직원 수는?"

▸ Alphabet 10-K: "Full-time employees: **183,323**"

STEP 3 · Final Synthesis

4개 Retrieval Result를 Input Query 기준으로 비교

계산: 매출 차 = $\$391.0B - \$350.0B = \$41B$ (Apple 우위)

직원 차 = $183,323 - 164,000 = 19,323$ 명 (Google 우위)

→ Apple의 매출이 \$41B 많고, 직원 수는 Google이 19,323명 많음

한 번의 Retrieval로 네 가지 근거를 모두 확보하기 어려워 Query Decomposition으로 보완한다.



② Query Decomposition

복합 질문을 독립적으로 검색할 수 있는 2~5개의 Subquery로 나눈다. 각 Subquery를 검색한 뒤 결과를 통합해 최종 답변을 생성한다.

벤치마크 결과

- ✓ LangChain 2024 benchmark: Multi-hop QA +15~25% 정확도
- ✓ UC Berkeley DecomposeRAG: 복합 질문 ~50% 개선
- ✓ 복합 질문을 여러 Retrieval로 나누는 일반적인 구현 패턴

한계

- ✗ Subquery 수만큼 Retrieval 비용 증가 + Final Synthesis 호출
- ✗ 직렬 실행 시 지연 증가, 병렬 실행 시 가장 느린 Retrieval에 좌우
- ✗ 단순 질문을 불필요하게 분해할 수 있음 ("환불은?" → 5개 분해 등)

구현

LangChain MultiQueryRetriever · QueryDecomposition (LCEL)

LlamaIndex SubQuestionQueryEngine

NVIDIA RAG Blueprint의 Query Decomposition 모듈



③ HyDE — Hypothetical Document Embeddings

짧은 질문과 긴 문서의 표현 차이가 큰 경우, LLM이 Hypothetical Document를 생성하고 이를 Embedding해 검색한다. 실제 답변을 만드는 것이 아니라 검색 대상 문서와 유사한 표현을 만드는 것이 목적이다.

Gao et al. (2022), "Precise Zero-Shot Dense Retrieval without Relevance Labels" · arXiv:2212.10496

예시 — 짧은 질문 "양자 컴퓨팅 원리"

Baseline — Input Query Embedding

Input Query

"양자 컴퓨팅 원리"

BGE Embedding

Short Query Embedding

Top-3 Retrieval Result

- ▶ "양자역학 입문 — 슈뢰딩거 방정식" (대학 강의노트) ← 키워드만 맞고 토픽 어긋남
- ▶ "컴퓨팅 산업 동향 2024" (보고서) ← 단어 매치, 양자 무관
- ▶ "원리 · 정의 모음집" (FAQ) ← 일반 명사가 통째로 끌어옴

짧은 Query와 긴 Document의 표현 차이로 관련 Chunk가 Top-K에서 누락될 수 있음.

HyDE

STEP 1 · Input Query

"양자 컴퓨팅 원리"

STEP 2 · Generate Hypothetical Document

질문에 대응하는
Hypothetical Document 생성

~150 tokens · ~\$0.0008

Hypothetical Document d'

"양자 컴퓨팅은 큐비트(qubit)를 기본 단위로 사용하며, 중첩(superposition) 과 얽힘(entanglement)을 통해 고전 비트로 불가능한 병렬 상태를 표현한다. Shor-Grover 알고리즘은 이를 이용해 인수분해·검색을 가속한다."

→ Corpus의 문서와 유사한 길이와 표현을 사용

STEP 3 · Embed d'

(Input Query 대신 d' Embedding 사용)

STEP 4 · Retrieve Top-3 from Corpus

- ★ Nielsen & Chuang ch.1: "Qubits, superposition, and the basic principles..."
- ★ IBM Research blog: "How a quantum computer works — entanglement explained"
- ★ MIT OCW lecture: "Why quantum: parallelism via superposition"

관련 Chunk 3건 검색 · Recall@3: 0 → 1.0

STEP 5 · Generate Final Response from Top-3 + Input Query

→ "큐비트, 중첩, 얽힘을 활용한 병렬 상태 표현이 핵심 원리이며, Shor-Grover 알고리즘으로 응용된다." (출처 Chunk 인용)



③ HyDE — Hypothetical Document Embeddings

짧은 질문과 긴 문서의 표현 차이가 큰 경우, LLM이 Hypothetical Document를 생성하고 이를 Embedding해 검색한다. 실제 답변을 만드는 것이 아니라 검색 대상 문서와 유사한 표현을 만드는 것이 목적이다.

Gao et al. (2022), "Precise Zero-Shot Dense Retrieval without Relevance Labels" · arXiv:2212.10496



작동 원리 — Dense Embedding은 의미가 유사한 표현을 가까이 배치한다. 짧은 Query와 긴 Document는 길이와 문체가 달라 Embedding 공간에서 멀어질 수 있다. HyDE는 **Hypothetical Document**를 생성해 Document와 유사한 표현으로 Retrieval한다. 생성 결과의 사실성보다 검색 표현의 정렬이 중요하다.

벤치마크 결과

- ✓ BEIR·TREC 등 10개 벤치마크에서 Unsupervised Contriever 대비 개선
- ✓ 한국어·일본어·스와힐리어 등 다국어 결과가 보고됨
- ✓ 별도 학습 데이터 없이 새 도메인에 적용 가능

한계

- ✗ Query당 LLM 호출 1번 (+100~500ms, ~\$0.001)
- ✗ LLM의 가정이 부정확하면 검색 품질이 낮아질 수 있음
- ✗ 이미 길고 구체적인 질문에서는 효과가 제한적

구현

LlamaIndex HyDEQueryTransform

LangChain HypotheticalDocumentEmbedder

Haystack HypotheticalDocumentEmbedder (deepset)

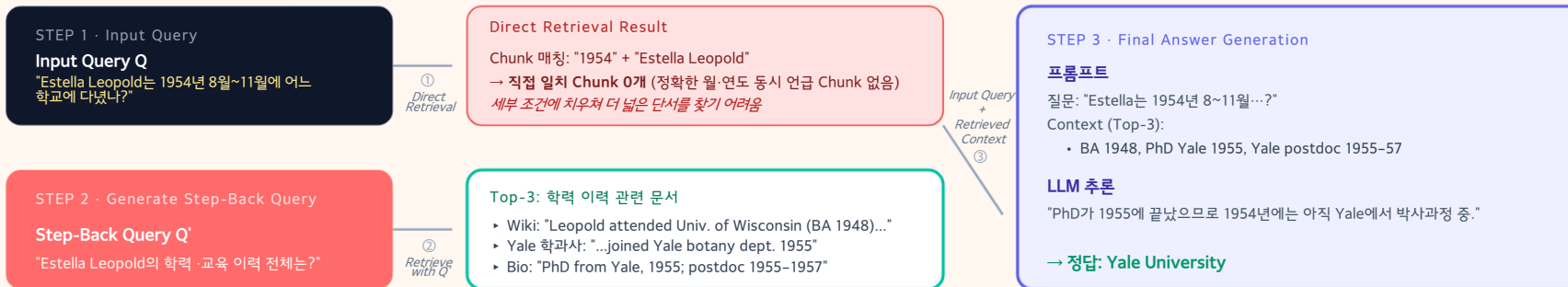


④ Step-Back Prompting

질문의 조건이 지나치게 구체적이면 한 단계 일반화한 질문을 먼저 만들고, 관련 원리나 배경 정보를 검색한 뒤 원래 질문에 적용한다. Google DeepMind가 2023년에 제안한 방식이다.

Zheng et al. (2024), "Take a Step Back: Evoking Reasoning via Abstraction" · ICLR 2024 · arXiv:2310.06117

예시 — 특정 시점의 학력 질문



작동 원리

- ① 원본 Q의 "1954년 8~11월"은 너무 좁아 Corpus에 정확히 매칭되는 Chunk가 없다.
- ② 시점을 제거한 Q'은 Estella의 "이력" 전반을 다루는 일반 문서에 강하게 매칭된다 (recall ↑).
- ③ LLM은 받은 이력 Chunk에서 1948 / 1955 / 1957 같은 구간 정보를 보고 "1954년에는 박사과정"이라 추론한다. 검색은 단서를 모으는 역할, 시점 매칭은 LLM에 맡긴다.

Step-Back Prompting의 효과가 제한적인 경우

- ▶ 이미 일반적인 질문("Estella는 누구인가?") — 더 추상화할 게 없음
- ▶ 고유 식별자의 정확 일치에 필요한 질문("Error TS-999가 뭐냐") — BM25가 더 적합

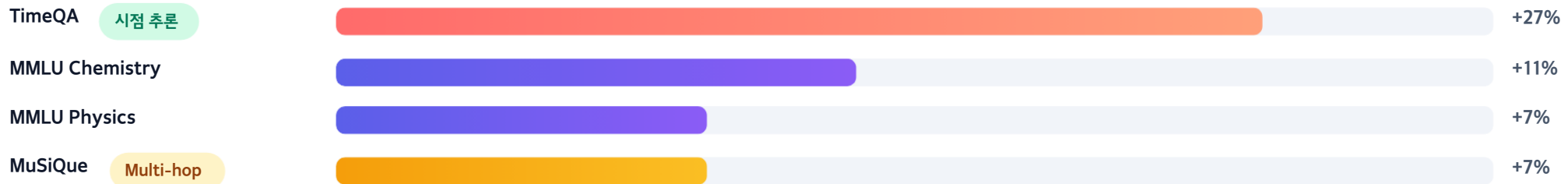


④ Step-Back Prompting

질문의 조건이 지나치게 구체적이면 한 단계 일반화한 질문을 먼저 만들고, 관련 원리나 배경 정보를 검색한 뒤 원래 질문에 적용한다. Google DeepMind가 2023년에 제안한 방식이다.

Zheng et al. (2024), "Take a Step Back: Evoking Reasoning via Abstraction" · ICLR 2024 · arXiv:2310.06117

벤치마크 결과 (PaLM 2-L)



막대는 Baseline 대비 Accuracy 개선 폭(%p)을 나타낸다. 시점 및 원리 추론이 필요한 Task에서 개선 폭이 컸다.

적합한 질문

- ✓ 시점·연도가 포함된 질문
- ✓ 특정 수치를 묻지만 "원리"가 필요한 STEM 문제
- ✓ 한 단계의 Multi-hop 추론

적합하지 않은 질문

- ✗ 이미 일반적인 질문 ("AI란?") — 더 추상화 불필요
- ✗ 고유명사 정확 매칭 ("Error TS-999") — BM25가 더 효과적
- ✗ 비용: LLM 호출 1번 추가

구현 [LangChain](#) [Step-Back Prompting Cookbook \(LCEL\)](#)

[Custom](#) [Few-shot Example 3~5개로 구현](#)

Query Transformation 비교

기법	적합한 질문	추가 LLM 호출	추가 지연	대표 결과
Query Rewriting	모호하거나 검색어가 부족한 질문	1회	+100~300ms	도메인별 평가 필요
Query Decomposition	여러 사실을 함께 요구하는 복합 질문	1회 + 검색 N회 + 합성 1회	+500ms~2s	+15~25% (Multi-hop QA, LangChain)
HyDE	짧은 Query와 긴 Document의 표현 차이	1회 (Hypothetical Document 생성)	+200~600ms	Unsupervised Contriever 대비 개선, 다국어 결과 보고
Step-Back Prompting	조건이 지나치게 구체적인 추론 질문	1회 (Step-Back Query 생성)	+200~500ms	TimeQA +27%, MMLU Phy/Chem +7~11%

 **적용 가이드** — 네 기법을 동시에 다 쓰지 말 것. 평가셋으로 "내 사용자 질문이 어떤 패턴이 많은가" 분석 후, 빈번한 약점 패턴 1~2개만 골라 적용. 비용/지연 vs 품질 트레이드오프를 매번 측정.

Hybrid Search — Dense Retrieval과 Sparse Retrieval의 결합

Dense Retrieval과 Sparse Retrieval(BM25)은 서로 다른 유형의 근거를 찾는다. RRF(Reciprocal Rank Fusion)는 두 Ranking을 결합하는 대표적인 Rank Fusion 방식이다.

Dense Retrieval과 Sparse Retrieval의 상호 보완성

Dense Retrieval

- ✓ 동의어와 의미적으로 유사한 표현에 강함 ("환불" ↔ "반품")
- ✓ Cross-lingual Retrieval 가능
- ✗ 고유명사, 약어, 코드의 정확 일치에 약함 ("TS-999")
- ✗ 빈도가 낮은 키워드에 둔감할 수 있음

Sparse Retrieval (BM25)

- ✓ 키워드의 정확 일치에 강함 ("TS-999")
- ✓ 별도 학습 없이 적용 가능
- ✗ 동의어와 의미적 유사성을 직접 반영하지 못함
- ✗ 한국어에서는 형태소 분석 품질의 영향을 받음

Dense Retrieval과 BM25는 상호 보완적이다. 공개 사례에서는 Hybrid Search가 Recall을 15~25% 높였고, Anthropic 실험에서는 BM25 추가 후 Failure Rate 감소 폭이 35%에서 49%로 확대됐다.

RRF (Reciprocal Rank Fusion) — 순위 결합 공식

Cormack, Clarke & Buettcher (SIGIR 2009)가 제안한 Rank Fusion 방식이다. 별도 학습 없이 적용할 수 있다.

RRF Formula

$$\text{RRF_score}(d) = \sum_{\text{retriever } r} 1 / (k + \text{rank}_r(d))$$

각 Retriever에서 문서 d 의 Rank를 받아 역수 합을 계산한다. k 는 안정화 상수이며 일반적으로 60에서 시작한다. 여러 Retriever에서 모두 상위에 있는 문서일수록 RRF Score가 높아진다.

k=60을 사용하는 이유

Cormack et al. (2009)은 여러 데이터셋에서 **k=60**이 안정적으로 작동한다고 보고했다. k 가 작으면 최상위 결과의 영향이 커지고, k 가 커지면 순위 간 차이가 완만해진다.

RRF가 안정적으로 작동하는 이유

Dense Retrieval의 Cosine Similarity와 BM25 Score는 범위가 다르다. 원점수를 직접 합산하면 한쪽 Retriever가 결과를 지배할 수 있지만, RRF는 **Rank만 사용**하므로 별도의 Score Normalization 없이 결합할 수 있다.

RRF 계산 예시

RRF Formula

계산 예시 (k=60)

질문: "고객이 환불 신청을 어떻게 하나요?". Dense와 BM25에서 각각 Top-10을 가져온 상황.

문서	Dense Retrieval Rank	BM25 Rank	RRF Formula	RRF Score	Final Rank
"환불 정책 v3"	2	1	$1/(60+2) + 1/(60+1)$ $= 0.01613 + 0.01639$	0.03252	🏆 1위
"반품 절차 안내"	1	5	$1/(60+1) + 1/(60+5)$ $= 0.01639 + 0.01538$	0.03178	🏆 2위
"지불 수단 종류"	8	2	$1/(60+8) + 1/(60+2)$ $= 0.01471 + 0.01613$	0.03084	🏆 3위
"FAQ 모음"	5	—	$1/(60+5)$ $= 0.01538$	0.01538	4위 (한쪽만)
"카드 결제 매뉴얼"	—	3	$1/(60+3)$ $= 0.01587$	0.01587	5위 (한쪽만)

두 Retriever에서 모두 상위에 있는 문서가 우선된다. 한쪽에서만 검색된 문서도 후보로 남지만 Rank는 낮아진다.

다른 Fusion 방식과 비교

방법	아이디어	장점	단점
RRF	순위 역수 합	단순, 튜닝 필요 없음, 다중 검색기 자연 확장	원점수의 크기 정보는 사용하지 않음
Weighted Sum	각 점수에 가중치 곱해 합산 $\alpha \cdot \text{dense} + (1-\alpha) \cdot \text{bm25}$	도메인별 Weight 조정 가능	Weight 설정과 Score Normalization 필요
CombSUM / CombMNZ	점수 합 (CombSUM), 중복 가중 (CombMNZ)	전통적인 Information Retrieval 방식	데이터셋에 따라 RRF보다 낮은 성능
Learned Fusion (LTR)	지도 학습으로 결합 가중치 학습	도메인 학습 데이터가 있으면 높은 성능 가능	학습 데이터와 별도 모델 운영 필요

구현

LangChain EnsembleRetriever (RRF 포함)

LlamaIndex QueryFusionRetriever (mode="reciprocal_rerank")

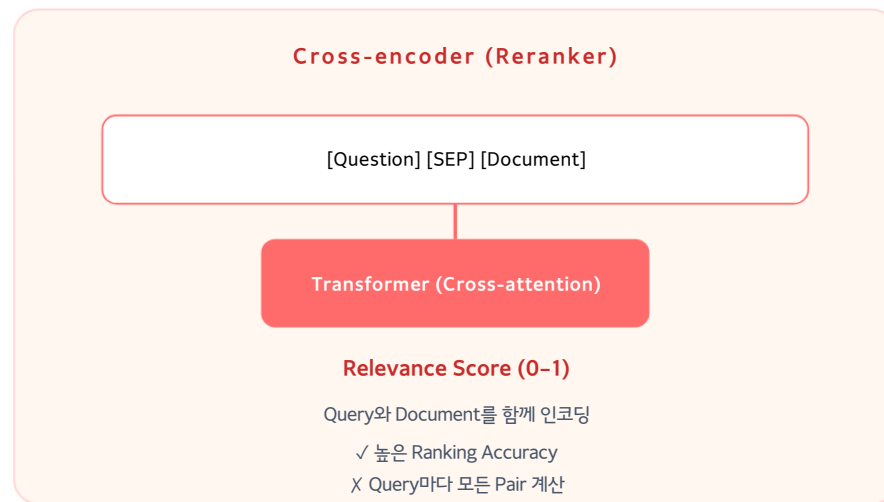
벡터 DB 내장. Weaviate · Qdrant · Elasticsearch 모두 hybrid 지원

 **적용 가이드** — Dense Retrieval Top-50 + BM25 Top-50 → RRF로 Top-30~50 결합 → Reranker로 Top-5~10 정제. 후보 수는 평가셋과 지연 목표에 맞춰 조정한다.

Reranking — 검색 후보의 순위를 정밀하게 조정

1차 Retriever는 빠르게 후보를 찾지만 상위 순위의 정밀도는 제한적이다. Reranker는 (Query, Chunk) 쌍의 관련성을 다시 계산해 후보 순위를 조정한다. Anthropic의 Contextual Retrieval 실험에서는 Reranker를 추가한 뒤 Retrieval Failure Rate가 2.9%에서 1.9%로 낮아졌다.

Bi-encoder와 Cross-encoder 비교



Bi-encoder로 Top-50~150 후보를 빠르게 검색한 뒤, Cross-encoder로 재정렬하는 2단계 구조가 일반적이다. Cross-encoder는 질문과 문서를 함께 인코딩하므로 정확도가 높지만 Query마다 후보 수만큼 계산이 필요하다.

Reranker 유형

- Cross-encoder
- Cohere Rerank API
- LLM-as-Reranker

Cross-encoder Reranker

BERT 또는 MiniLM 계열 모델을 (질문, 문서, 관련성) 데이터로 Fine-tuning한 방식이다. sentence-transformers, BGE-Reranker, mxbai-rerank 계열이 대표적이다.

- ✓ BGE-Reranker-v2-m3: 한국어 포함 다국어 · 568M Parameters · Open Model
- ✓ Cross-encoder/ms-marco-MiniLM: 영어 중심, 가벼움 (수십~수백 MB)
- ✓ mxbai-rerank-large-v2: 영어·중국어·다국어 지원

장점

- ✓ Self-hosting 가능 · 데이터 외부 전송 없음
- ✓ GPU에서 Pair당 약 10~50ms
- ✓ Domain Fine-tuning 가능

한계

- ✗ GPU 인프라 필요
- ✗ 모델 서빙과 모니터링 필요
- ✗ 긴 입력은 Truncation 필요

구현

sentence-transformersCrossEncoder('BAAI/bge-Reranker-v2-m3')

LlamaIndexSentenceTransformerRerank

LangChainCrossEncoderReranker

Reranker 유형

Cross-encoder

Cohere Rerank API

LLM-as-Reranker

Cohere Rerank API

Cohere가 제공하는 Managed Reranker다. 별도 모델 인프라 없이 API 호출로 후보 문서를 재정렬할 수 있으며, Anthropic의 Contextual Retrieval 실험에도 사용됐다.

- ✓ **rerank-v3.5**: 최신 모델, 100+ 언어, 4096 토큰 입력
- ✓ **rerank-3-nimble**: 3~5× 빠르지만 약간 낮은 정확도
- ✓ BEIR Benchmark에서 NDCG 개선을 보고(Cohere 공개 결과)

```
# 사용 예시
import cohere
co = cohere.Client(api_key=KEY)
results = co.rerank(
    model=
    "rerank-v3.5",
    query=
    "환불 정책",
    documents=[chunk1, chunk2,...],
    top_n=
    10)
```

Cross-encoder

Cohere Rerank API

LLM-as-Reranker

Cohere Rerank API

장점

- ✓ 별도 인프라 없이 적용 가능
- ✓ 다국어 지원 · 한국어 포함
- ✓ 지연: 수십~수백 ms

한계

- ✗ 외부 API 사용 · 데이터 전송 및 요청 비용 발생
- ✗ 모델 내부 조정과 Fine-tuning이 제한적
- ✗ Offline 환경에서 사용 불가

구현

Cohere API `co.rerank(model="rerank-v3.5",...)`

LangChain. `CohereRerank`

LlamaIndex. `CohereRerank`

Reranker 유형

Cross-encoder

Cohere Rerank API

LLM-as-Reranker

LLM-as-Reranker

LLM에 후보 Chunk와 질문을 함께 제공하고 관련성 순서를 생성하도록 하는 방식이다. **Pointwise**는 Chunk별 점수를 계산하고, **Listwise**는 후보 목록 전체의 순서를 한 번에 결정한다.

- ✓ Listwise (RankGPT, RankZephyr): 전체 Candidate List를 한 번에 재정렬
- ✓ Pointwise: Chunk별 Relevance Score를 계산하며 병렬 처리 가능
- ✓ Large LLM: 별도 Fine-tuning 없이 적용 가능

Listwise 프롬프트 예시

"""다음 10개 문서를 사용자 질문과의 관련성 순으로
가장 관련 있는 것부터 정렬해 줘.
답변은 문서 번호 리스트만, 예: [3, 7, 1, 9,...]

질문: {query}

문서 1: {chunk_1}
문서 2: {chunk_2}...
문서 10: {chunk_10}
"""

Cross-encoder

Cohere Rerank API

LLM-as-Reranker

LLM-as-Reranker

장점

- ✓ 별도 학습 없이 적용 가능
- ✓ 관련성 판단 이유를 함께 생성 가능
- ✓ 다국어 및 도메인 전이에 유연

한계

- ✗ Query당 LLM 호출 비용이 큼
- ✗ Latency가 수 초까지 증가할 수 있음
- ✗ Context Window 제약 (긴 Chunk가 많을수록 Input Token 증가)

구현

Custom 단순 프롬프트로 가능

LlamaIndex LLMRerank · RankGPTRerank

오픈소스 RankZephyr · RankVicuna

Reranker 유형별 비교

유형	정확도	지연 (50 chunks)	비용	인프라	언제
Cross-encoder (BGE, mxbai)	높음	50~500ms (GPU)	Self-hosting 비용	GPU 필요	데이터 외부 전송이 제한된 환경
Cohere Rerank	높음	100~300ms	\$2/1K queries	API	운영 편의성과 성능의 균형. Anthropic 실험에 사용된 Reranker
LLM-as-Reranker (Claude/GPT-4)	높음 · 모델에 따라 다름	1~5초	높음 · LLM 호출 비용	API	낮은 QPS에서 품질과 설명이 중요한 경우

 **적용 가이드** — 초기 검증에는 Managed API인 **Cohere Rerank**를 사용할 수 있다. 데이터 외부 전송이 제한된 환경에서는 **BGE-Reranker-v2-m3** 같은 Self-hosted Model을 검토한다. 1차 Retriever에서 충분한 Candidate를 확보한 뒤 Top-5~10으로 줄이는 구성이 일반적이다.

Contextual Retrieval — Chunk에 문서 맥락 추가

Anthropic은 2024년 9월 Contextual Retrieval을 공개했다. 인덱싱 단계에서 각 Chunk 앞에 문서 내 위치와 주제를 설명하는 짧은 Context를 추가한다. 공개 실험에서는 Contextual BM25 결합 시 Retrieval Failure Rate가 49% 감소했고, Reranking까지 추가했을 때 67% 감소했다.

문제 — Chunk 분할로 문서 맥락이 사라짐

ACME 2024 Q2 SEC 10-Q 보고서

Item 2. 매출 분석

ACME의 2024 Q2 북미 매출은 \$1.2B...

전 분기 \$1.16B 대비 분석:

"The company's revenue grew by 3% over the previous quarter."

... 다음 섹션 ...

Chunk

Context 없는 Chunk

"The company's revenue grew by 3% over the previous quarter."

사용자 질문:

"ACME 2024 Q2 매출은?"

→ Chunk에 ACME, Q2, 2024 정보가 없음
Retrieval 실패

+ LLM-generated
Context

Contextualized Chunk

[LLM-GENERATED CONTEXT]

"이 Chunk는 ACME Corp의
2024 Q2 SEC 10-Q 보고서의
매출 분석 섹션에서 발췌"

"The company's revenue grew
by 3% over the previous quarter."

→ "ACME 2024 Q2 매출" Query에 매칭

동작 방식 — Indexing 단계에서 Context 추가

Prompt Template

```
<document>
{{WHOLE_DOCUMENT}}
</document>
```

Here is the chunk we want to situate within the whole document:

```
<chunk>
{{CHUNK_CONTENT}}
</chunk>
```

Please give a short succinct context to situate this chunk within the overall document for the purposes of improving search retrieval of the chunk. Answer only with the succinct context.

Claude 3 Haiku로 Chunk당 50~100 Token의 Context를 생성해 Prefix로 추가

비용 구조와 Prompt Caching

같은 문서의 여러 Chunk에 Context를 생성하면 문서 본문이 반복 입력된다. **Prompt Caching**을 사용하면 반복 입력 비용을 크게 줄일 수 있다.

\$1.02 / 1M 문서 토큰

가정: 800-Token Chunk, 8K-Token Document, 50-Token Prompt, 100-Token Context.
100M Document Tokens 기준 약 \$102.

Anthropic 공개 결과 — Failure Rate @ Top-20

Dataset: Codebase, Fiction, ArXiv, Science Papers · Embedding: Gemini Text 004 · Metric: 1 — Recall@20



Cohere Rerank로 Top-150을 Top-20으로 재정렬했다. Voyage와 Gemini Embedding Model에서 개선 폭이 컸다.

Contextual Retrieval — 조합 효과와 적용 기준

단계별 효과가 누적된다

Embeddings → Contextual Embeddings → Contextual BM25 → Reranking 순으로 단계를 추가할 때마다 Retrieval Failure Rate가 낮아졌다.

LLM에 전달할 문서 수

공개 실험에서는 Reranker로 정제한 Top-20을 LLM에 전달했을 때 Top-10 또는 Top-5보다 좋은 결과가 나왔다. 다만 문서 수가 지나치게 많아지면 Lost-in-the-Middle 문제가 발생할 수 있다.

Embedding Model에 따른 차이

Anthropic 실험에서는 Voyage와 Gemini Embedding Model에서 개선 폭이 컸다. 한국어 도메인에서는 실제 사용 모델별로 효과를 다시 측정해야 한다.

구현

[Anthropic Cookbook](#) [contextual-embeddings guide](#)[LlamaIndex](#) [llama_index.packs.contextual_retrieval](#)[LangChain](#) [Runnable + Prompt Caching](#) 별도 구성

적용 조건 — 문서 제목, 섹션, 시점 정보가 Chunk 분할 과정에서 자주 사라지는 도메인에서 우선 검토한다. 도입 전 실제 Corpus와 Embedding Model 조합으로 Retrieval 지표를 비교하고, Prompt Caching 조건을 반영해 Indexing 비용을 별도로 산정한다.

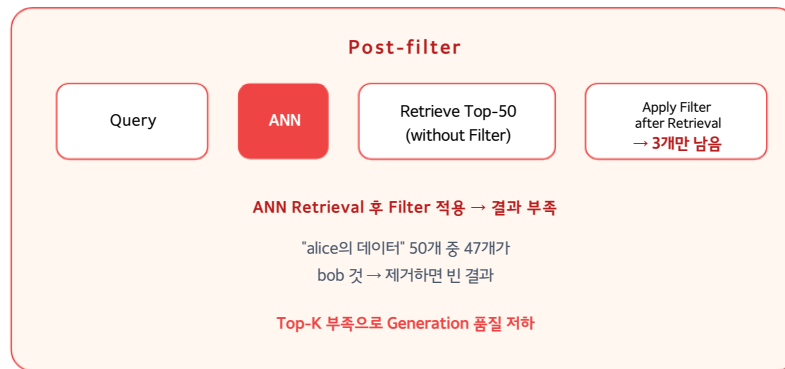
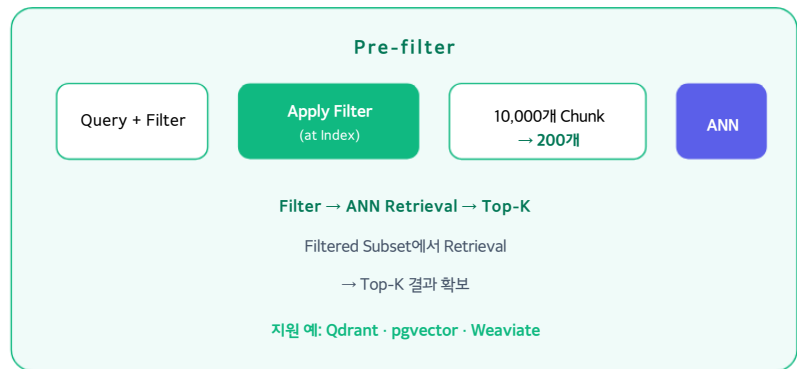
Metadata Filtering · Parent-Child Retrieval · Small-to-Big Retrieval

Metadata Filtering으로 검색 범위를 제한하고, Parent-Child Retrieval과 Small-to-Big Retrieval로 작은 Chunk의 검색 정밀도와 큰 Context의 정보량을 함께 활용한다.

① Metadata Filtering

Chunk에 { "user": "alice", "category": "환불", "date": "2024-Q3", "lang": "ko" }와 같은 Metadata를 저장하고, Retrieval 조건으로 사용한다.

Pre-filter와 Post-filter 비교



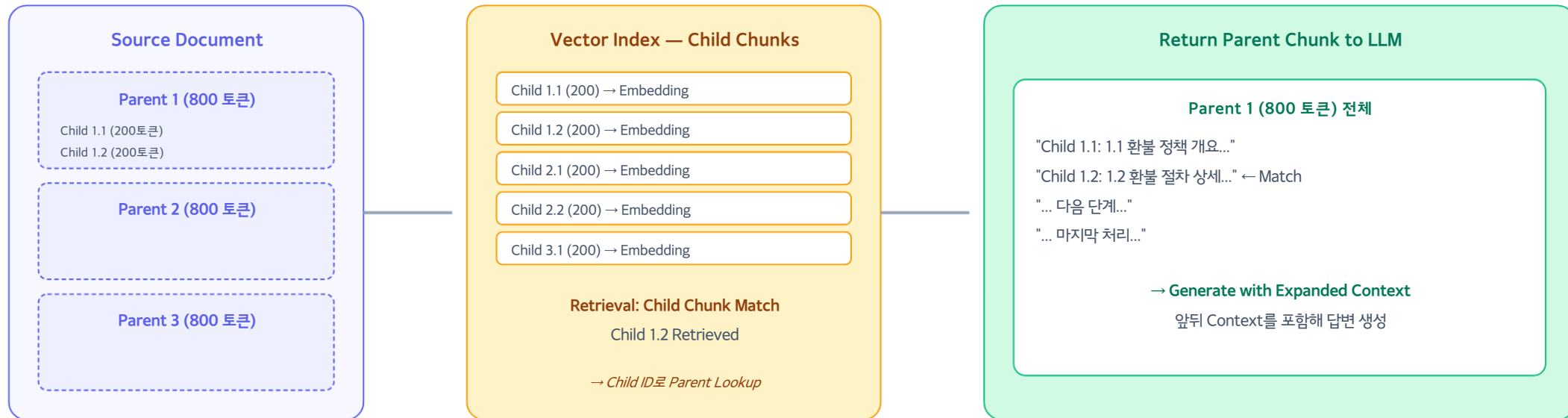
⚠ **주의** — 사용자·권한·카테고리 조건은 가능하면 **Pre-filter**로 적용한다. **Post-filter**는 ANN Retrieval 이후 후보를 제거하므로 Top-K가 부족해질 수 있다. Vector DB 선택 시 Filterable HNSW 또는 동등한 Pre-filter 지원 여부를 확인한다.

Parent-Child Retrieval · Small-to-Big Retrieval

② Parent-Child Retrieval / Small-to-Big Retrieval

작은 Chunk는 Retrieval Precision이 높지만 앞뒤 Context가 부족할 수 있다. 큰 Chunk는 Context가 풍부하지만 여러 주제를 하나의 Embedding에 담게 된다. Child Chunk로 검색하고 연결된 Parent Chunk를 반환해 두 장점을 결합한다.

Index Small, Return Large



Structural Retrieval Patterns

주요 구현 패턴

Sentence Window Retrieval

문장 단위로 Indexing하고, 검색된 문장의 주변 $\pm 3 \sim 5$ 문장을 함께 반환한다.

 LlamaIndex `SentenceWindowNodeParser`

Parent Document Retriever

Child Chunk와 Parent Chunk를 명시적으로 매핑한다. Child는 Vector Store, Parent는 Document Store에 저장한다.

 LangChain `ParentDocumentRetriever`

Auto-Merging Retriever

같은 Parent에 속한 여러 Child Chunk가 검색되면 Parent Chunk로 자동 병합한다.

 LlamaIndex `AutoMergingRetriever`

Chunk Size Guide

Parent 500~2000 토큰

Child 100~500 토큰

비율 Parent $\div$ Child $\approx 4 \sim 10$

 **적합한 경우** — 문서가 길고 답이 한 문장에 들어있지만 그 문장만으로는 LLM에 필요한 맥락이 부족한 경우 (법률 조항·기술 문서). **비용: 인덱스가 커짐(작은 Chunk N배 + 큰 Chunk)**. 효과는 도메인에 따라 다르므로 평가셋으로 비교 필수.

Multi-hop RAG — 단순 Retrieval 구조의 한계

앞선 기법을 조합해도 여러 사실을 순차적으로 검색하고 연결해야 하는 질문에는 한계가 있다. Multi-hop 벤치마크에서는 Full Context 대비 Chunked Context의 정확도가 약 30%p 낮아지는 사례가 보고된다. 여기서는 원인과 보완 접근을 함께 살펴본다.

Multi-hop 질문

한 문서나 Chunk에 답이 완결되어 있지 않고, 두 개 이상의 근거를 순서대로 검색하고 연결해야 하는 질문이다.

Single-hop과 Multi-hop 비교

SINGLE-HOP (단순 RAG에 적합)

질문

"Apple의 2024 Q3 매출은?"

Chunk 1개에 답

"Apple Q3 매출: \$85.8B"

Retrieval 1회 → 근거 1개 → 답변

Hybrid Search와 Reranking으로 안정적인 성능 확보 가능

Multi-hop (반복 Retrieval 필요)

질문

"Tim Cook 자녀의 모교는?"

Hop 1: Tim Cook 자녀는?

(여러 문서 탐색 필요)

Hop 2: 그 사람의 모교는?

(1번 답에 의존)

Retrieval N회 + Intermediate Reasoning + Synthesis

NovelHopQA: Full Context 대비 약 30%p 하락

Multi-hop RAG 벤치마크 결과

벤치마크	출처	특징	관찰된 결과
HotpotQA	Yang et al. 2018 (EMNLP)	Wikipedia 기반 2-hop 질문 약 90K개 · Supporting Facts 제공	Full-context Fine-tuned Model: 약 70 EM. RAG 방식은 보통 10~20%p 낮음.
MultiHop-RAG	Tang & Yang 2024 (COLM 2024) (arXiv 2401.15391)	뉴스 기반의 복잡한 Multi-hop Query로 구성된 RAG Evaluation Benchmark	기존 RAG 시스템의 Multi-hop Retrieval과 답변 생성 성능이 낮다고 보고했다.
MuSiQue	Trivedi et al. 2022	2~4 Hop 추론 질문	RAG 방식의 Accuracy는 20~50% 범위. Step-Back Prompting 적용 시 약 7% 개선.
NovelHopQA	2024+	장편 소설에서 Multi-hop 추론	Chunked Context는 Full Context 대비 Accuracy가 약 30%p 낮아 Retrieval Bottleneck을 보여준다.

6 Multi-hop & RAG LIMITS

Multi-hop RAG — 단순 RAG가 약한 이유

단순 RAG가 Multi-hop에 약한 이유

1. 검색 단계 간 의존성

두 번째 Retrieval이 첫 번째 Retrieval의 결과에 의존하는 경우, 한 번의 검색으로는 필요한 근거를 모두 확보하기 어렵다.

2. 근거의 분산

질문에 필요한 사실이 서로 다른 Chunk나 문서에 분산된다. 모든 Hop의 근거가 Top-K 안에 함께 포함되어야 LLM이 최종 답을 만들 수 있다.

3. Embedding 기반 검색의 한계

Embedding 기반 검색은 질문과 의미적으로 가까운 문서를 우선한다. 중간 엔티티를 아직 알 수 없는 질문에서는 다음 Hop에 필요한 문서가 Query와 직접 유사하지 않아 검색에서 누락될 수 있다.

Multi-hop RAG — 보완 접근

보완 접근과 한계

접근	방법	효과	한계
Query Decomposition	질문을 Subquery로 분해해 각각 검색	독립적인 Hop에서 +15~25% 개선 사례가 보고됨.	Hop 1 답이 Hop 2에 필요한 의존적인 Multi-hop에는 한계
Iterative RAG	1차 검색 → 부분 답 → 그 답으로 2차 검색 → 반복	의존적인 Hop을 처리할 수 있음 · ReAct, Self-RAG, FLARE 등	반복 횟수만큼 비용과 지연 증가 · 종료 조건 설계 필요
GraphRAG	문서에서 엔티티·관계를 추출해 지식 그래프 생성, 그래프 탐색	FalkorDB 보고: 5+ 엔티티 질문에서 vector RAG 대비 3.4× 정확도.	LLM 기반 엔티티 추출로 인덱싱 비용과 운영 복잡도가 증가.
Agentic RAG	LLM이 도구처럼 검색을 호출, 필요할 때 반복·재계획	복잡한 Retrieval Plan을 처리할 수 있음.	비용과 지연이 크며 실행 경로 디버깅이 어려움
Long-context LLM	Source Document(200K+)를 한 번에 LLM에 주입	문서가 Context Window 안에 들어오면 Full Context가 대안	긴 입력은 비용과 Lost-in-the-Middle 문제를 고려해야 함



핵심 결론

- ✓ 단순 RAG는 **한 번의 Retrieval 후 답변을 생성하는 구조**이므로 Single-hop 질문에 적합하다.
- ✓ Multi-hop 질문은 Query Decomposition, Iterative RAG, GraphRAG, Agentic RAG로 보완할 수 있지만, 모든 유형에 공통으로 적용되는 해법은 없다.
- ✓ MultiHop-RAG와 같은 벤치마크에서는 최신 접근도 개선 폭이 제한적이며, 현재도 활발히 연구되는 영역이다.
- ✓ 적용 전 전체 질문 중 Multi-hop 비율을 측정한다. **5% 미만이면 우선순위를 낮추고, 20% 이상이면 본격 도입을 검토**한다. 중간 구간은 평가셋에서 비용 대비 효과를 비교한다.

RAG가 구조적으로 어려운 과제

수치 계산과 집계

전체 문서의 값을 집계해야 하는 질문은 Top-K Retrieval만으로 처리하기 어렵다. Text-to-SQL이나 구조화 데이터 접근이 필요하다.

시간 기준 검색과 추론

Vector Search만으로 최신성을 판단하기 어렵다. 날짜 Metadata와 Time-aware Retrieval을 함께 사용해야 한다.

상충하는 정보

서로 충돌하는 문서가 검색될 수 있으므로 Version과 Source Reliability 정보를 Context에 포함해야 한다.

추론과 계획이 필요한 과제

RAG는 근거를 제공하는 구성 요소다. 진단·계획처럼 복잡한 과제에는 별도 Reasoning과 Tool Use가 필요하다.

MultiHopRAG 250문항 파일럿 평가

Tang & Yang (COLM 2024)의 MultiHopRAG 데이터셋 250문항을 사용해 Retrieval 구성과 Chunking Strategy별 상대 성능을 비교했다.



평가 설정

- ✓ Corpus: 609개 뉴스 기사 (HuggingFace yixuantt/MultiHopRAG) · Evaluation: 전체 2,556개 Query 중 총화 표본 250문항 — comparison 84 · inference 80 · temporal 57 · null 29
- ✓ Retriever: Top-K=6 · Embedding Model: BAAI/bge-small-en-v1.5 (384-dim) · BM25: rank_bm25 · Hybrid Search: BM25 + Dense Retrieval, RRF (k_each=30, c=60)
- ✓ Models: Generator / Decomposer / Judge = Claude Haiku 4.5 (local CLI pool, concurrency 8)
- ✓ Metrics: Hit@6(any) · Recall@6 · MRR · Haiku-as-Judge(correct/partial/incorrect) · Accuracy = (correct + 0.5×partial) / n

결과 ① — Retrieval 구성 비교 (Sentence-aware Chunking 고정)

Retrieval 구성	Overall Accuracy	Hit@6	Recall@6	MRR	Comparison (n=84)	Inference (n=80)	Temporal (n=57)	Null (n=29)
Dense Retrieval BGE-small	37.8%	91.0%	52.1%	0.725	24.4%	40.0%	26.3%	93.1%
Hybrid Search (BM25 + Dense Retrieval, RRF) BM25 + Dense Retrieval + RRF	50.6% +12.8%p	95.0%	63.6%	0.779	36.3%	57.5%	38.6%	96.6%
+ Query Decomposition Hybrid Search + Query Decomposition	61.6% +11.0%p	95.0%	70.8%	0.737	53.0%	70.0%	44.7%	96.6%

7 PILOT EVALUATION

Retrieval 구성별 결과

Hybrid Search: +12.8%p

Dense Retrieval만 사용했을 때보다 BM25를 RRF로 결합한 **Hybrid Search**에서 Recall@6가 52.1%에서 63.6%로 상승했다. Overall Accuracy도 37.8%에서 50.6%로 12.8%p 개선됐다.

Query Decomposition: +11.0%p

Query Decomposition을 추가하면 Inference Accuracy가 57.5%에서 70.0%로 상승했다. Recall@6도 63.6%에서 70.8%로 7.2%p 개선되어, Subquery별 Retrieval이 필요한 근거를 더 많이 확보한 것으로 나타났다.

Temporal 질문은 여전히 낮은 성능

세 구성의 Temporal Accuracy는 모두 44.7% 이하였다. "가장 최근", "X 이후 최초"와 같은 질문에는 날짜 Metadata와 Time-aware Retrieval을 별도로 설계해야 함을 시사한다.

✓ Abstention on Null Queries

세 구성 모두 Null Query의 93~97%에서 "Insufficient information"으로 답변을 보류했다. 근거가 부족할 때 Abstain하도록 Prompt와 Evaluation 기준을 설계하는 것이 중요하다.

결과 ② — Chunking Strategy 비교 (Hybrid Search 고정)

동일한 Retrieval·Generation Pipeline에서 5가지 Chunking Strategy를 비교했다. Retrieval 구성은 Hybrid Search + RRF로 고정했다.

Chunking Strategy	# Chunks (609 Articles)	Overall Accuracy	Recall@6	MRR	특징
Fixed-size Chunking (Small) 400자, overlap 80	19,828	46.4%	61.7%	0.764	작은 Chunk로 Multi-hop 근거가 분산됨
Fixed-size Chunking (Large) 1200자, overlap 200	6,484	49.2%	61.8%	0.804	MRR은 가장 높지만 Overall Accuracy는 중간 · 불필요한 Context 포함
Recursive LangChain식, target 800	10,090	50.4%	64.7%	0.777	문장·단락 경계 우선 · Recall@6가 가장 높음
Sentence-aware ~800자, 문장 단위 packing	9,527	50.6%	63.6%	0.779	Overall Accuracy가 가장 높음 · 경계 보존이 안정적
Semantic Chunking (BGE) 인접 문장 cosine ≥ 0.55 merge	11,272	48.2%	64.3%	0.777	뉴스에서는 주제 전환이 잦아 추가 비용 대비 개선이 제한적

Chunking Strategy 비교

Chunking Strategy별 Overall Accuracy (Hybrid Search, 250문항)



1. 문장·단락 경계 보존이 가장 안정적

Recursive Chunking과 Sentence-aware Chunking이 각각 50.4%, 50.6%로 가장 높은 Overall Accuracy를 보였다. 문장과 단락 경계가 분명한 문서에서는 경계를 보존하는 방식이 안정적이었다.

2. Semantic Chunking의 효과는 도메인에 따라 달라짐

Semantic Chunking은 Recursive Chunking보다 낮은 Overall Accuracy를 보였다. 뉴스 기사처럼 주제가 자주 전환되는 문서에서는 의미 변화 지점이 Multi-hop 근거를 분리할 수 있다. 효과는 논문이나 매뉴얼처럼 장문 구조가 일정한 도메인에서 다시 확인해야 한다.

파일럿 결과 해석과 한계

3. 지나치게 작은 Chunk는 근거를 분산시킴

Fixed-size Chunking (400자)은 Overall Accuracy가 46.4%로 가장 낮았다. Top-K=6에서 Chunk가 지나치게 작으면 Multi-hop에 필요한 근거가 여러 Chunk로 분산될 가능성이 커진다.

4. 이번 파일럿에서는 Retrieval 구성이 더 큰 영향을 보임

Chunking Strategy 5종의 Overall Accuracy 범위는 46.4~50.6%(4.2%p), Retrieval 구성 3종의 범위는 37.8~61.6%(23.8%p)였다. 이번 파일럿에서는 Chunking 미세 조정보다 Hybrid Search와 Query Decomposition의 영향이 더 컸다.

⚠ 파일럿 범위와 해석 기준

- ✕ 250문항은 전체 2,556문항의 표본이며 Full Evaluation이 아니다. 표본 크기가 커지면 결과가 달라질 수 있다.
- ✕ Judge와 Generator가 모두 Haiku이므로 평가 편향 가능성이 있다. 별도 Judge Model을 사용한 재평가가 필요하다.
- ✕ Contextual Retrieval과 Reranking은 비용과 시간 제약으로 파일럿에서 제외했다. 관련 수치는 Anthropic과 Cohere의 공개 자료를 사용했다.
- ✕ 코드: `RAGwithVibeCoding/src/` · 결과 JSON: `results/pilot_results*.json`

Session 3 요약 — 기법별 적용 조건

기법별 효과, 비용, 적용 조건

기법	주요 효과	추가 지연	비용 추가	적용 우선순위
Query Rewriting	모호 질문 명료화	+100~300ms	LLM 호출 1회/질의	낮음 · 질문 유형 확인 후 적용
Query Decomposition	복합 질문 +15~25%	+500ms~2s	LLM N+2회/질의	중간 · Multi-hop 비율에 따라
HyDE	짧은 질문에 효과	+200~600ms	LLM 1회/질의	낮음 · 짧은 Query가 많은 도메인
Step-Back Prompting	추론·시점 질문 +7~27%	+200~500ms	LLM 1회/질의	중간 · 추론형 QA 도메인
Hybrid Search + RRF	Recall +15~25%	+10~50ms	BM25 인덱스 추가	높음 · 우선 검토

기법별 효과 · 비용 · 적용 조건

기법별 효과, 비용, 적용 조건

기법	주요 효과	추가 지연	비용 추가	적용 우선순위
Reranking	Top-K Ranking 품질 개선	+50~500ms	Cohere API 또는 Self-hosted GPU	높음 · 우선 검토
Contextual Retrieval	Failure Rate 49~67% 감소	0 (인덱싱 시만)	\$1.02/M 토큰 (Haiku)	높음 · Context 의존 도메인
Metadata Filtering	검색 범위 축소	0 또는 감소 가능	Indexing 단계에서 Metadata 저장	높음 · 권한·카테고리 Filter가 필요한 경우
Parent-Child Retrieval	정밀 Retrieval + 확장된 Context	+5~20ms	Child Index + Parent Store	중간 · 긴 문서
Iterative RAG / Agentic RAG	Multi-hop 질문 보완	+수초	높음 · 반복 LLM 호출	낮음 · 복합 질문 비율이 높은 경우

권장 도입 순서

1단계 — 기본 Retrieval 품질 개선

- ✓ Hybrid Search + RRF — Dense Retrieval + BM25
- ✓ Reranking — Cohere Rerank API 또는 BGE-Reranker
- ✓ Metadata Filtering — Pre-filter 지원 DB

→ Hybrid Search Top-50~150 → Reranker Top-5~10

2단계 — 도메인별 보완

- ✓ Contextual Retrieval — Document Context 의존도가 높은 Chunk
- ✓ Query Decomposition — 복합 질문 비율이 높은 경우
- ✓ Parent-Child Retrieval — 긴 Document

→ 평가셋에서 개선 폭을 측정한 뒤 결정

3단계 — 복합 질문 대응

- ✓ HyDE / Step-Back Prompting — 특정 질문 유형
- ✓ Agentic RAG / Iterative RAG — Multi-hop 질문 비율이 높은 경우
- ✓ GraphRAG — Entity-Relation 중심 Corpus

→ 비용 대비 효과를 검증한 뒤 적용



Session 3 요약 — Hybrid + Reranker + Contextual은 우선 검토할 수 있는 기본 조합이다. 나머지 기법은 도메인과 질문 패턴에 따라 선택한다. **Multi-hop은 여전히 어려운 영역**이므로 평가셋으로 효과를 검증해야 한다.

KNOWLEDGE CHECK

10문제 퀴즈 — 고급 RAG 기법

Session 3의 주요 기법과 파일럿 결과를 10문제로 확인한다. 각 문항의 선택지를 먼저 검토한 뒤 해설을 확인한다.



진행 방법 — 5지선다 또는 OX 형식. 해설에는 "왜 다른 보기는 틀린가"까지 포함. 8문제 이상 맞히면 Session 4(평가·운영)로 넘어갈 준비 완료.

10문제 퀴즈 — 고급 RAG 기법

1 주요 기법 중 Indexing 단계에서 적용되는 것은?

A. Query Rewriting

B. HyDE

C. Contextual Retrieval

D. Reranking

E. Step-Back Prompting

정답: C Contextual Retrieval은 Indexing 단계에서 각 Chunk에 문서 맥락을 설명하는 Context를 Prefix로 추가한다.

Query Rewriting, HyDE, Step-Back Prompting은 Query 시점에, Reranking은 Retrieval 이후에 적용된다.

10문제 퀴즈 — 고급 RAG 기법

2 다음 중 HyDE가 가장 적합한 질문은?

- A. “양자 컴퓨팅 원리”처럼 짧고 키워드 중심인 질문
- B. 여러 회사의 매출과 직원 수를 함께 비교하는 질문
- C. 특정 연도와 월을 포함한 세부 이력 질문
- D. “Error TS-999”처럼 고유 식별자의 정확 일치가 필요한 질문

정답: A HyDE는 짧은 Query와 긴 문서의 표현 차이를 줄이기 위해 Hypothetical Document를 생성해 Retrieval한다.

B에는 Query Decomposition, C에는 Step-Back Prompting, D에는 BM25가 더 적합하다.

10문제 퀴즈 — 고급 RAG 기법

3 MultiHopRAG 파일럿에서 Dense Retrieval을 Hybrid Search로 변경했을 때 Overall Accuracy 개선 폭은?

A. +3%p

B. +7%p

C. +12.8%p (37.8% → 50.6%)

D. +20%p

E. +30%p

정답: C Hybrid Search 적용 후 Recall@6는 52.1%에서 63.6%로, Overall Accuracy는 37.8%에서 50.6%로 상승했다.

이 파일럿에서는 BM25와 RRF를 추가해 12.8%p의 개선이 관찰됐다.

10문제 퀴즈 — 고급 RAG 기법

4 RRF(Reciprocal Rank Fusion)에서 일반적으로 사용하는 상수 k의 기본값은?

A. 1

B. 10

C. 60 (Cormack et al. 2009)

D. 100

E. 1000

정답: C (k=60)

$\text{score}(d) = \sum 1/(k + \text{rank}(d))$.

k가 작으면 최상위 Rank의 영향이 커지고, 크면 Rank간 차이가 완만해진다.

RRF는 원점수가 아니라 Rank를 결합하므로 Dense Retrieval과 BM25의 Score Scale 차이를 피할 수 있다.

10문제 퀴즈 — 고급 RAG 기법

5 Bi-encoder와 Cross-encoder의 차이에 대한 설명 중 잘못된 것은?

- A. Bi-encoder는 질문과 문서를 독립적으로 인코딩한다
- B. Cross-encoder는 질문과 문서를 함께 인코딩한다
- C. Cross-encoder는 문서 Embedding을 미리 계산할 수 있다
- D. Bi-encoder로 후보를 검색한 뒤 Cross-encoder로 재정렬할 수 있다

정답: C Cross-encoder는 Query와 문서를 함께 인코딩하므로 문서 표현을 Query와 독립적으로 미리 계산할 수 없다.
따라서 전체 Corpus가 아니라 1차 Retriever가 만든 후보 집합에 적용한다.

10문제 퀴즈 — 고급 RAG 기법

6 OX. Anthropic의 Contextual Retrieval은 각 Chunk에 문서 맥락을 설명하는 Context를 Prefix로 추가하며, 공개 실험에서 Retrieval Failure Rate를 49~67% 낮췄다.

A. ☐ 맞다 (True)

B. ☒ 틀리다 (False)

정답: A (True) Contextual Embeddings 단독으로 35%, Contextual BM25 결합 시 49%, Reranker까지 추가했을 때 67%의 Retrieval Failure Rate 감소가 보고됐다.

Chunk에 없는 문서 제목, 섹션, 시점 정보를 보완해 Retrieval 표현을 개선한다.

10문제 퀴즈 — 고급 RAG 기법

7 Step-Back Prompting의 효과가 가장 제한적인 질문은?

- A. 특정 연도와 시점이 포함된 질문
- B. 원리를 적용해야 하는 STEM 문제
- C. 한 단계의 Multi-hop 추론이 필요한 질문
- D. 이미 일반적인 질문

정답: D Step-Back Prompting은 질문을 한 단계 일반화하므로, 이미 일반적인 질문에서는 추가 변환의 이점이 적다.

고유 식별자의 정확 일치에 필요한 질문도 BM25가 더 적합하다.

10문제 퀴즈 — 고급 RAG 기법

8 NovelHopQA에서 Chunked Context가 Full Context보다 약 30%p 낮은 Accuracy를 보인 결과가 의미하는 것은?

- A. LLM 자체의 추론 능력만 부족하다
- B. 필요한 Multi-hop 근거를 함께 검색하는 과정이 병목이다
- C. Embedding Model의 크기만 늘리면 해결된다
- D. Chunk를 더 작게 만들수록 유리하다

정답: B 같은 LLM이라도 Chunked Context에서는 필요한 Hop의 근거가 함께 검색되지 않을 수 있다.

Query Decomposition, Iterative RAG, GraphRAG가 이를 보완하는 접근이다.

10문제 퀴즈 — 고급 RAG 기법

9 파일럿에서 Dense Retrieval, Hybrid Search, Query Decomposition 모두 45% 미만이었던 질문 유형은?

A. Comparison Query

B. Inference Query

C. Temporal Query

D. Null Query

정답: C (Temporal) Temporal Accuracy는 각각 26.3%, 38.6%, 44.7%였다.

날짜 Metadata와 Time-aware Retrieval을 별도로 설계해야 함을 시사한다.

10문제 퀴즈 — 고급 RAG 기법

10 Query Decomposition 또는 Agentic RAG를 본격적으로 검토할 기준으로 가장 적절한 것은?

A. 사용자 트래픽이 일정 수준 이상일 때

B. 모델 가격이 낮아졌을 때

C. Multi-hop 또는 복합 질문 비율이 20% 이상일 때

D. 평균 답변 길이가 길어졌을 때

정답: C 먼저 질문 분포를 측정하고, Multi-hop 비율이 20% 이상이면 본격 도입을 검토한다.

5% 미만이면 우선순위를 낮추고, 중간 구간은 평가셋에서 비용 대비 효과를 비교한다.



QUIZ

10문제 퀴즈 — 고급 RAG 기법



스코어카드

- ✓ 9~10개 정답 — 핵심 개념을 충분히 이해한 수준
- ✓ 7~8개 정답 — 틀린 문항의 관련 절을 다시 확인
- ✓ 5~6개 정답 — Query Understanding과 Contextual Retrieval을 다시 확인
- ✓ 4개 이하 — Query Understanding, Hybrid Search, Contextual Retrieval부터 다시 확인